



Relational Database Design

Mr. S.K.Patil

Relational Database Design

Constraints –

- **Constraints** are a very **important feature** in a **relational model**.
- The relational model supports the theory of **constraints on attributes or tables**.
- Constraints are useful because they **allow a designer to specify the semantics of data** in the database.
- Constraints are the **rules that force DBMSs to check that data satisfies the semantics**.
- In modeling the design of the relational database we can put some restrictions like **what values are allowed to be inserted in the relation**, and **what kind of modifications and deletions are allowed in relations**.
- These are the **restrictions we impose on the relational database**.

Relational Database Design

Constraints –

- **Database Constraints** can be categorized into 3 main categories –
 - Constraints that are **applied in the data model** are called **Implicit Constraints**.
 - Constraints that are **directly applied in the schemas of the data model**, by **specifying** them in **DDL**. These are called **Schema-based Constraints or Explicit Constraints**.
 - Constraints that **can not be directly applied in the schemas** of the data model. These are called **Application based or Semantic Constraints**.

Relational Database Design

Integrity Constraints –

- Integrity constraints **ensure** that **changes made to the database by authorized users do not result in a loss of data consistency.**
- Integrity constraints **guard against accidental damage to the database.**
- **Examples** of integrity constraints are –
 - A **student name cannot be null.**
 - **No two students** can have the **same student ID.**
 - The **budget** of a department **must be greater than \$0.00.**
- In general, an **integrity constraint** can be an **arbitrary predicate pertaining to the database.**
- However, **arbitrary predicates** may be **costly to test.**
- Thus, most database systems allow one to **specify integrity constraints that can be tested with minimal overhead.**

Relational Database Design

Integrity Constraints –

- Integrity constraints are usually identified as **part of the database schema design process**, and **declared as part of the create table** command used to create relations.
- However, **integrity constraints** can also be **added to an existing relation** by **using** the **command alter table table-name add constraint**, where constraint can be any constraint on the relation.
- When such a command is executed, the system first ensures that the relation satisfies the specified constraint.
- If it does, the constraint is added to the relation; if not, the command is rejected.

Relational Database Design

Integrity Constraints –

➤ Constraints on a Single Relation –

- In **addition to** the **primary-key constraint**, there are a number of **other** ones that can be **included in the create table** command.
- The allowed integrity constraints include –
 - **not null** - prohibits the insertion of a null value for the attribute.
 - **unique** - no two tuples in the relation can be equal on all the listed attributes.
 - **check(<predicate>)** - to ensure that **attribute values satisfy specified conditions**

Relational Database Design

Types of Integrity Constraints –

- There are several **types of constraints** available in DBMS and they are –
 - **Domain Constraints**
 - **Entity Integrity Constraints**
 - **Referential Integrity Constraints**
 - **Key Constraints**

Relational Database Design

Types of Integrity Constraints –

➤ Domain Constraint -

- It can be defined as the **definition of a valid set of values for an attribute**.
- The **data type of domain** includes string, character, integer, time, date, currency, etc.
- The **value of the attribute must be available** in the **corresponding domain**.



ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

Relational Database Design

Types of Integrity Constraints –

➤ Entity Integrity Constraint -

- It states that **primary key value can't be null**.
- This is because the **primary key** value is **used to identify individual rows** in relation and **if the primary key has a null value**, then **we can't identify those rows**.
- A **table can contain a null value other than the primary key field**.

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

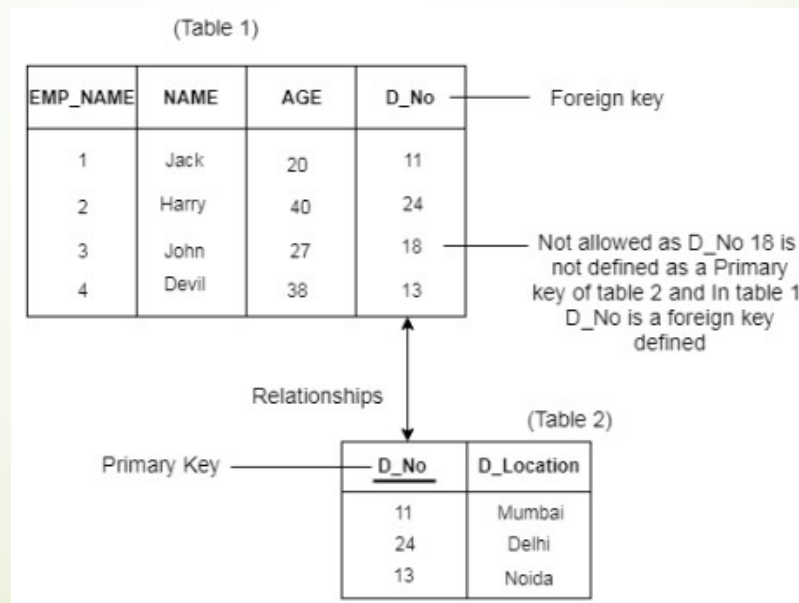
Not allowed as primary key can't contain a NULL value

Relational Database Design

Types of Integrity Constraints –

➤ Referential Integrity Constraint -

- It is **specified between two tables**.
- In this, if a **foreign key in table 1 refers to the primary key of table 2**, then **every value of the foreign key in table 1 must be null or be available in table 2**.



Relational Database Design

Types of Integrity Constraints –

➤ Key Constraint -

- **Keys** are the **entity set** that is **used to identify an entity within its entity set uniquely**.
- An **entity set** can have **multiple keys**, but out of which **one key will be the primary key**.



ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1002	Morgan	8 th	22

Not allowed. Because all row must be unique

Relational Database Design

Relational Database Design –

- The **Relational Database** model was **proposed by E. F. Codd in 1969**.
- The Relational Database Model is **based on branch of mathematics** called **set theory and predicate logic**.
- The **idea** behind to design the Relational Database model is that the **database consist of series of unordered table or relation that can be manipulated using non-procedural operations that return tables**.
- A relational **DBMS must use** its **relational facilities** exclusively **to manage and interact with the database**.
- **When designing a database, we need to decide how to choose a best model in the real world and how this best model is fitted in the database.**
- When designing a database, we need to decide **which tables to create, what columns they will contain**, as well as the **relationships between the tables**.

Relational Database Design

Relational Database Design –

➤ Goals -

- Design should ensure that **all database operations will be efficiently performed.**
- DBMS **should not perform expensive consistency checks.**
- **No data redundancy** should be there.

Relational Database Design

Features of Good Relational Database Design –

- The **primary feature** of a relational database is its **primary key**, which is a **unique identifier assigned to every record in a table**.
- **e.g. registration number**. It makes **every record unique**, facilitating the **storage of data in multiple tables**, and **every table** in a relational database **must have a primary key field**.
- Another key feature of relational databases is their **ability to hold data over multiple tables**.
- This feature **overcomes the limitations of simple flat file databases** that can **only have one table**.
- The **database records stored in a table** are **linked to records in other tables** by the **primary key**.
- The **primary key** can **join the table** in a **one-to-one relationship, one-to-many relationship or many-to-many relationship**.

Relational Database Design

Features of Good Relational Database Design –

- Relational databases **enable users to delete, update, read and create data entries in the database tables.**
- This is **accomplished through structured query language**, or SQL, which is based on relational algebraic principles.
- **SQL** also **enable users to manipulate and query data in a relational database.**
- Relational **tables follow various integrity rules** that **ensure the data stored in them is always accessible and accurate.**
- The **rules coupled with SQL enable users to easily enforce transaction and concurrency controls** with the **guarantee of data integrity.**

Relational Database Design

Functional Dependency –

- **Functional Dependency** is a **constraint** that **determines the relation of one attribute to another attribute in a DBMS.**
- It **specifies** the **relationship between two sets of attributes** where **one attribute determines the value of another attribute.**
- It **helps** to **maintain the quality of data** in the database.
- It plays a vital role to **find the difference between good and bad database design.**
- It is **denoted by an arrow '→'.**
- The **functional dependency of X on Y** is represented by **$X \rightarrow Y$.**
- The **left side of FD** is known as a **determinant**, the **right side** of the production is known as a **dependent.**

Relational Database Design

Functional Dependency –

➤ Example –

- Assume we have **an employee table** with attributes: **Emp_Id, Emp_Name, Emp_Address**.
- Here **Emp_Id** attribute can **uniquely identify** the **Emp_Name** attribute of employee table because if we know the Emp_Id, we can tell that employee name associated with it.
- Functional dependency can be written as:

Emp_Id → Emp_Name

- We can say that **Emp_Name is functionally dependent on Emp_Id**.
- Que – roll_no, name, dept_name, dept_building ????????

Relational Database Design

Functional Dependency –

- **Armstrong's axioms/properties of functional dependencies –**
 - **Axioms, or rules of inference**, provide a **simpler technique** for **reasoning about functional dependencies**.
 - We can use the following **three rules** to find **logically implied functional dependencies**.
 - **Reflexivity**: If **Y is a subset of X**, then **$X \rightarrow Y$** holds by **reflexivity rule**.
Example, **$\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$** is valid.
 - **Augmentation**: If **$X \rightarrow Y$** is a **valid dependency**, then **$XZ \rightarrow YZ$** is also valid by the **augmentation rule**.
Example, **$\{\text{roll_no}, \text{name}\} \rightarrow \text{dept_building}$** is valid, hence **$\{\text{roll_no}, \text{name}, \text{dept_name}\} \rightarrow \{\text{dept_building}, \text{dept_name}\}$** is also valid.
 - **Transitivity**: If **$X \rightarrow Y$** and **$Y \rightarrow Z$** are both **valid dependencies**, then **$X \rightarrow Z$** is also valid by the **Transitivity rule**.
Example, **$\text{roll_no} \rightarrow \text{dept_name}$ & $\text{dept_name} \rightarrow \text{dept_building}$** , then **$\text{roll_no} \rightarrow \text{dept_building}$** is also valid.

Relational Database Design

Functional Dependency –

➤ Types of Functional Dependencies in DBMS –

There are **four types of functional dependencies** -

- Trivial Functional Dependency
- Non - Trivial Functional Dependency
- Multivalued Functional Dependency
- Transitive Functional Dependency

Relational Database Design

Functional Dependency –

➤ Types of Functional Dependencies in DBMS –

➤ Trivial Functional Dependency –

- In Trivial functional dependency, a **dependent is always a subset of the determinant**. i.e. If $X \rightarrow Y$ and Y is the subset of X .
- In other words, a functional dependency is called trivial if the **attributes on the right side are the subset of the attributes on the left side of the functional dependency**.
- It occurs when an **attribute or set of attributes on the left-hand side** of a functional dependency arrow ($\rightarrow$) **already determines the attributes on the right-hand side without any extra information**.
- $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent **name** is a subset of determinant set **$\{\text{roll_no}, \text{name}\}$** .
- Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also trivial functional dependency.

Relational Database Design

Functional Dependency –

➤ Types of Functional Dependencies in DBMS –

➤ Non-Trivial Functional Dependency –

- In Non-trivial functional dependency, the **dependent is strictly not a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is not a subset of X .**
- **roll_no** $\rightarrow$ **name** is a non-trivial functional dependency, since the dependent **name** is **not a subset** of determinant **roll_no**.
- **{roll_no, name}** $\rightarrow$ **age** is also a non-trivial functional dependency, since **age** is **not a subset** of **{roll_no, name}**

Relational Database Design

Functional Dependency –

➤ Types of Functional Dependencies in DBMS –

➤ Multivalued Functional Dependency –

- In **Multivalued functional dependency**, entities of the dependent set are **not dependent on each other**. i.e. If $a \rightarrow \{b, c\}$ and there exists **no functional dependency** between **b** and **c**.
- $\text{roll_no} \rightarrow \{\text{name}, \text{age}\}$ is a multivalued functional dependency, since the dependents **name & age** are **not dependent** on each other.

Relational Database Design

Functional Dependency –

➤ Types of Functional Dependencies in DBMS –

➤ Transitive Functional Dependency –

- In transitive functional dependency, **dependent is indirectly dependent on determinant**. i.e. If $a \rightarrow b$ & $b \rightarrow c$, then according to axiom of transitivity, $a \rightarrow c$.
- $enrol_no \rightarrow dept$ and $dept \rightarrow building_no$. Hence, according to the axiom of transitivity, $enrol_no \rightarrow building_no$ is a **valid functional dependency**.

Relational Database Design

Functional Dependency –

➤ Types of Functional Dependencies in DBMS –

➤ Fully Functional Dependency –

- If X and Y are an attribute set of a relation, **Y is fully functional dependent on X**, if **Y is functionally dependent on X but not on any proper subset of X**.
- In the relation **$ABC \rightarrow D$** , attribute **D is fully functionally dependent on ABC** and **not on any proper subset of ABC**. That means subsets of ABC like AB, BC, A, B, etc. cannot determine D.
- **Example** – Consider **Supply table** with attributes – **supplier_id, item_id, price**. **Neither supplier_id nor item_id can uniquely determine the price but both supplier_id and item_id together can do so.**
- So we can say that price is fully functionally dependent on { supplier_id, item_id }.

Relational Database Design

Functional Dependency –

➤ Types of Functional Dependencies in DBMS –

➤ Partial Functional Dependency –

- A functional dependency $X \rightarrow Y$ is a partial dependency if **Y is functionally dependent on X and Y can be determined by any proper subset of X.**
- For example, we have a relationship **$AC \rightarrow B$, $A \rightarrow D$, and $D \rightarrow B$.**
- If we compute the **closure of $\{A^+\}=ADB$ (Closure - set of attributes which can be functionally determined from it).**
- Here **A is alone capable of determining B**, which means **B is partially dependent on AC.**
- Example – Student table with attributes name, roll_no, course

Relational Database Design

Functional Dependency –

➤ Advantages of Functional Dependencies –

- Functional dependencies having numerous applications in the field of database management system. Here are some applications listed below:

➤ Data Normalization –

- The **process of organizing data in a database** in order **to minimize redundancy and increase data integrity**.
- Functional dependencies play an **important part in data normalization**.
- With the help of functional dependencies we are able **to identify the primary key, candidate key in a table** which in turns helps in normalization.

Relational Database Design

Functional Dependency –

➤ Advantages of Functional Dependencies –

➤ Query Optimization –

- With the help of functional dependencies we are able to **decide the connectivity between the tables and the necessary attributes** need to be projected **to retrieve the required data** from the tables.
- This helps in **query optimization** and **improves performance**.

➤ Consistency of Data –

- Functional dependencies **ensures the consistency of the data by removing any redundancies or inconsistencies** that may exist in the data.
- It ensures that the **changes made in one attribute does not affect inconsistency in another set of attributes** thus it **maintains the consistency of the data in database**.

Relational Database Design

Functional Dependency –

➤ Advantages of Functional Dependencies –

➤ Data Quality Improvement –

- Functional dependencies ensure that the **data in the database to be accurate, complete and updated.**
- This helps to **improve the overall quality of the data**, as well as it **eliminates errors and inaccuracies** that might occur during data analysis and decision making.
- Thus functional dependency helps in **improving the quality of data in database.**

Relational Database Design

Functional Dependency –

- **Data Redundancy** - keeping data in two or more places within a database or data storage system.
- **Data Consistency** - state of data in which all copies or instances are the same across all systems and databases
- **Data Integrity** - concept and process that ensures the accuracy, completeness, consistency, and validity of an organization's data.
- **Completeness of data** - dataset contains all the necessary elements and observations for a given purpose or analysis.
- **Atomicity** – either all occur or none occur.
- **Durability** - safeguarding data from loss or corruption in the event of an outage or failure
- **Isolation** - separation of data to keep it safe from external cyberattacks and internal threats, and it can have many forms.

Relational Database Design

Normalization –

- **Normalization** is the **process of organizing data in a database** in order to **minimize redundancy and increase data integrity**.
- Normalization is **used to minimize the redundancy** from a relation or set of relations.
- It is also **used to eliminate undesirable characteristics** like Insertion, Update, and Deletion Anomalies.
- Normalization **divides the larger table into smaller and links them using relationships**.
- It is a **multi-step process** that **sets the data into tabular form** and **removes the duplicated data** from the relational tables.
- The **purpose of Normalization** in SQL is to **eliminate redundant (repetitive) data and ensure data is stored logically**.

Relational Database Design

Normalization –

➤ Purpose of Normalization -

- The **main purpose** of database normalization is to **avoid complexities, eliminate duplicates, and organize data in a consistent way.**
- In normalization, the **data is divided into several tables linked together with relationships.**
- **Database administrators are able to achieve these relationships by using primary keys, foreign keys, and composite keys.**
- It is **used to remove database anomalies** from the relational table.
- It **avoids duplicate data or no repeating groups into a table.**
- It **reduces the chances for anomalies** to occur in a database.

Relational Database Design

Normalization –

➤ Types of Anomalies -

- Following are the **types of anomalies** that **make the table inconsistency, loss of integrity, and redundant data.**

➤ Data Redundancy –

- It **occurs** in a relational database **when two or more rows or columns have the same value or repetitive value** leading to unnecessary utilization of the memory.

➤ Insert Anomaly –

- It **occurs** in the relational database **when some attributes or data items are to be inserted into the database without existence of other attributes.**
- **Example** - In the Student table, if we want **to insert a new courseID**, we **need to wait until the student enrolled in a course.**

Relational Database Design

Normalization –

➤ Types of Anomalies -

➤ Update Anomalies –

- The anomaly **occurs when duplicate data is updated only in one place and not in all instances**. Hence, it **makes our data or table inconsistent state**.
- If the value of one attribute is updated in one table then it also reflected in other tables also where it is used.

➤ Delete Anomalies –

- An anomaly **occurs in a database table when some records are lost or deleted from the database table due to the deletion of other records**.
- **Example** - if we want to **remove name ABC from the Student table**, it **also removes his address, course and other details** from Student table.

Relational Database Design

Normalization –

- Types of Normalization -
 - First Normal Form (1NF)
 - Second Normal Form (2NF)
 - Third Normal Form (3NF)
 - Boyce and Codd Normal Form (BCNF)
 - Fourth Normal Form (4NF)
 - Fifth Normal Form (5NF)

Relational Database Design

Normalization –

➤ First Normal Form (1NF)-

- For a **table to be in the First Normal Form**, it should **follow the following 4 rules :**
 - It should **only have single(atomic) valued attributes/columns**.
 - **Values** stored in a column **should be of the same domain**.
 - **All the columns** in a table should **have unique names**.
 - And the **order in which data is stored should not matter**.

Relational Database Design

Normalization –

➤ First Normal Form (1NF)-

- **Example** - If we have an **Employee table** in which we store the **employee information** along with the **employee skillset**, the table will look like this:

emp_id	emp_name	emp_mobile	emp_skills
1	John Tick	9999957773	Python, JavaScript
2	Darth Trader	8888853337	HTML, CSS, JavaScript
3	Rony Shark	7777720008	Java, Linux, C++

- The above table has 4 columns: **All columns have different names**, All **columns hold values of same type** like emp_name has all the names, emp_mobile has all the contact numbers, etc., The **order in which we save data doesn't matter**.
- But the **emp_skills** column **holds multiple comma-separated values**, while **as per the First Normal form, each column should have a single value**.

Relational Database Design

Normalization –

➤ First Normal Form (1NF)-

➤ Example –

- Hence the **above table fails to pass** the **First Normal form**.
- There are **two ways to fix** this:
 - **Remove** the **emp_skills** column from the **Employee table** and **keep** it in **some other table**.
 - **Add multiple rows** for the employee and each row is linked with one skill.

Relational Database Design

Normalization –

➤ First Normal Form (1NF)-

➤ Example –

➤ Create Separate tables for Employee and Employee Skills -



emp_id	emp_name	emp_mobile
1	John Tick	9999957773
2	Darth Trader	8888853337
3	Rony Shark	7777720008

emp_id	emp_skill
1	Python
1	JavaScript
2	HTML
2	CSS
2	JavaScript
3	Java
3	Linux
3	C++

Relational Database Design

Normalization –

➤ First Normal Form (1NF)-

➤ Example –

- **Add Multiple rows for Multiple skills** – We can simply add multiple rows to add multiple skills. This will lead to repetition of the data, but that can be handled as we further Normalize our data

emp_id	emp_name	emp_mobile	emp_skill
1	John Tick	9999957773	Python
1	John Tick	9999957773	JavaScript
2	Darth Trader	8888853337	HTML
2	Darth Trader	8888853337	CSS
2	Darth Trader	8888853337	JavaScript
3	Rony Shark	7777720008	Java
3	Rony Shark	7777720008	Linux
3	Rony Shark	7777720008	C++

Relational Database Design

Normalization –

➤ Second Normal Form (2NF)-

➤ For a table to be in the Second Normal Form -

➤ It should be in the First Normal form.

➤ It should not have Partial Dependency.

➤ What is Partial Dependency?

➤ When a table has a primary key that is made up of two or more columns, then all the columns(not included in the primary key) in that table should depend on the entire primary key and not on a part of it.

➤ If any column(which is not in the primary key) depends on a part of the primary key then we say we have Partial dependency in the table.

Relational Database Design

Normalization –

➤ Second Normal Form (2NF)-

➤ Example -

- If we have two tables **Students** and **Subjects**, to store student information and information related to subjects.

student_id	student_name	branch
1	Akon	CSE
2	Bkon	Mechanical

subject_id	subject_name
1	C Language
2	DSA
3	Operating System

Relational Database Design

Normalization –

➤ Second Normal Form (2NF)-

➤ Example -

- And we have another **table Score** to **store the marks scored by students in any subject.**

student_id	subject_id	marks	teacher_name
1	1	70	Miss. C
1	2	82	Mr. D
2	1	65	Mr. Op

- In the above table, the **primary key** is **student_id + subject_id**, because **both these information are required to select any row of data.**
- But **in the Score table**, we have a **column teacher_name**, which **depends on the subject information** or just the subject_id, so we **should not keep that information in the Score table.**

Relational Database Design

Normalization –

➤ Second Normal Form (2NF)-

➤ Example -

- The **column teacher_name** should be in the **Subjects table**. And then the **entire system will be Normalized** as per the Second Normal Form.

Updated **Subject** table:

subject_id	subject_name	teacher_name
1	C Language	Miss. C
2	DSA	Mr. D
3	Operating System	Mr. Op

Updated **Score** table:

student_id	subject_id	marks
1	1	70
1	2	82
2	1	65